DA5402 — MLOps

Assignment 6: The Orchestrated Web Scraper Pipeline

Building a Fault-Tolerant Data Orchestration Pipeline with Apache Airflow

Muhammed Fiyas
DA25M018

April 6, 2026

Abstract

This report documents the design, implementation, and testing of an automated Web-to-DB orchestration pipeline built using Apache Airflow 3.1.8. The pipeline monitors a directory for incoming CSV files containing target URLs, scrapes HTML and image assets from those URLs using BeautifulSoup, stores metadata in a SQLite database, and sends real-time email notifications for three distinct pipeline events: successful batch collection, broken link detection, and dry pipeline timeout. The system employs CeleryExecutor with Redis as a message broker, a custom Airflow Pool for concurrency control, and dynamic task mapping to process URLs in parallel while respecting rate limits.

Contents

1 Introduction **3**

2 System Architecture **3**
2.1 Infrastructure Overview . . . 3
2.2 DAG Architecture . . . 3

3 Implementation Details **4**
3.1 Task 1: FileSensor . . . 4
3.2 Task 2: Dry Pipeline Alert . . . 4
3.3 Task 3: Parse CSV . . . 4
3.4 Task 4: Scrape URL (Dynamic Task Mapping) . . . 5
3.5 Task 5: Store and Alert . . . 5
3.6 Tasks 6 & 7: Email Alerts . . . 5

4 Concurrency Control — Airflow Pool **6**

5 Airflow Connections **6**

6 Test Results **7**
6.1 Test 1 — Successful 3-URL Scrape . . . 7
6.2 Test 2 — Broken Link Alert . . . 7
6.3 Test 3 — Dry Pipeline Alert . . . 8
6.4 Collection Statistics Email . . . 9

7 Robustness & Error Handling **9**
7.1 Retry Logic . . . 9
7.2 Error Handling in Scraping . . . 9
7.3 Deduplication . . . 10

8 Conclusion **10**

A Repository Structure **11**

B AI Disclosure **11**

1 Introduction

Modern data engineering pipelines require robust orchestration to handle variable workloads, failures, and notifications without manual intervention. This assignment implements a production-grade "Web-to-DB" pipeline using Apache Airflow, addressing the core MLOps challenge of automating data collection workflows.

The pipeline satisfies the following core requirements:

- **Automated triggering** via a FileSensor that monitors an input directory
- **Concurrent scraping** throttled by an Airflow Pool (3 slots)
- **Persistent storage** of scraped metadata in SQLite with deduplication
- **Three email alert types**: dry pipeline, broken links, and collection stats
- **Fault tolerance** via retries with exponential backoff

2 System Architecture

2.1 Infrastructure Overview

The pipeline runs entirely within Docker containers managed by Docker Compose. The infrastructure consists of seven services as shown in Table 1.

Table 1: Docker Services

Service	Image	Role
airflow-apiserver	apache/airflow:3.1.8	REST API & Web UI
airflow-scheduler	apache/airflow:3.1.8	DAG scheduling
airflow-worker	apache/airflow:3.1.8	Task execution
airflow-dag-processor	apache/airflow:3.1.8	DAG file parsing
airflow-triggerer	apache/airflow:3.1.8	Deferred task triggers
postgres	postgres:16	Airflow metadata DB
redis	redis:7.2-bookworm	Celery message broker

2.2 DAG Architecture

The pipeline DAG (`web_scraper_pipeline`) consists of 8 tasks with the following dependency structure:

```
# FileSensor timeout path
wait_for_csv >> dry_pipeline_alert    # trigger_rule="all_skipped"

# Normal execution path
wait_for_csv >> db_init >> urls >> scraped >> store_result
store_result >> broken_alerts
store_result >> stats_email
```

Listing 1: Task dependency chain

Figure 1: DAG graph view in Airflow UI

3 Implementation Details

3.1 Task 1: FileSensor

The `FileSensor` monitors the `data/input/` directory for any `.csv` file. It uses `soft_fail=True` so that on timeout it marks itself as *Skipped* rather than *Failed*, which then triggers the dry pipeline alert via `trigger_rule="all_skipped"`.

```
wait_for_csv = FileSensor(
    task_id="wait_for_csv",
    filepath="/opt/airflow/data/input/*.csv",
    poke_interval=30,     # check every 30 seconds
    timeout=120,          # 2 min for testing (43200 = 12hrs production)
    mode="poke",
    soft_fail=True,       # SKIPPED on timeout, not FAILED
    fs_conn_id="fs_default",
)
```

Listing 2: FileSensor configuration

3.2 Task 2: Dry Pipeline Alert

Triggered only when `wait_for_csv` is skipped. Uses `smtplib` directly to send email via Gmail SMTP with STARTTLS on port 587.

```
@task(task_id="dry_pipeline_alert", trigger_rule="all_skipped")
def dry_pipeline_alert():
    # Sends email when no CSV detected within timeout window
    with smtplib.SMTP('smtp.gmail.com', 587) as server:
        server.starttls()
        server.login(smtp_user, smtp_password)
        server.sendmail(from_addr, to_addr, msg.as_string())
```

Listing 3: Dry pipeline alert task

3.3 Task 3: Parse CSV

Reads URLs from the most recently modified CSV file in the input directory. Returns the URL list via XCom for consumption by downstream tasks.

3.4 Task 4: Scrape URL (Dynamic Task Mapping)

The most critical task — uses Airflow's **dynamic task mapping** (`scrape_url.expand()`) to create one task instance per URL. All instances are assigned to `scraping_pool` to limit concurrent executions.

```
@task(
    task_id="scrape_url",
    pool="scraping_pool",           # max 3 concurrent (pool size)
    retries=3,
    retry_delay=timedelta(minutes=2),
    retry_exponential_backoff=True, # 2min -> 4min -> 8min
)
def scrape_url(url: str):
    headers = {'User-Agent': 'Mozilla/5.0 (compatible; AirflowScraper
   /1.0)'}
    response = requests.get(url, timeout=10, headers=headers)
    soup = BeautifulSoup(response.text, "html.parser")
    # Extract title and image links...

# One task per URL     dynamic task mapping
scraped = scrape_url.expand(url=urls)
```

Listing 4: Dynamic task mapping with pool

AI Attribution: The BeautifulSoup image URL extraction logic with relative/absolute URL handling was developed with AI assistance. See Appendix.

3.5 Task 5: Store and Alert

Inserts successful scrape results into SQLite using `INSERT OR IGNORE` for deduplication. Collects failed URLs for the broken link alert.

```
CREATE TABLE IF NOT EXISTS scraped_pages (
    id           INTEGER PRIMARY KEY AUTOINCREMENT,
    url          TEXT UNIQUE,          -- enforces deduplication
    title        TEXT,
    html_length  INTEGER,
    image_count  INTEGER,
    image_links  TEXT,
    status_code  INTEGER,
    scraped_at   TEXT
)
```

Listing 5: SQLite schema

3.6 Tasks 6 & 7: Email Alerts

Two downstream alert tasks run in parallel after `store_and_alert`:

- **send_broken_link_alerts**: Sends HTML table of failed URLs (404, timeout, connection errors) if any exist
- **send_collection_stats**: Sends DB summary email when total page count reaches the configured threshold (default: 3 for testing, 10 for production)

4 Concurrency Control — Airflow Pool

An Airflow Pool named `scraping_pool` with 3 slots was created via the Airflow UI (Admin → Pools). This prevents the pipeline from overwhelming target servers or consuming excessive network resources when processing large URL lists.

Figure 2: scraping_pool configuration in Airflow UI

When a CSV with more than 3 URLs is processed, excess `scrape_url` tasks queue until a slot becomes available, as illustrated below:

Slot	Status	Task
Slot 1	Running	scrape_url[0]
Slot 2	Running	scrape_url[1]
Slot 3	Running	scrape_url[2]
—	Queued	scrape_url[3]
—	Queued	scrape_url[4]

5 Airflow Connections

Two Airflow Connections were configured via Admin → Connections:

Table 2: Airflow Connections

Conn ID	Type	Purpose	Used By
`smtp_default`	Email	Gmail SMTP credentials	Email alert tasks
`fs_default`	File (path)	Filesystem root `/`	FileSensor

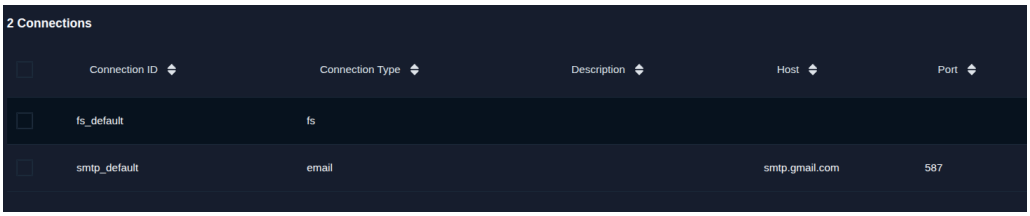


Figure 3: Airflow Connections in UI

SMTP credentials are stored securely via environment variable (`${SMTP_PASSWORD}`) in `docker-compose.yaml`, loaded from a gitignored `.env` file.

6 Test Results

6.1 Test 1 — Successful 3-URL Scrape

Input CSV:

```
url
https://books.toscrape.com
https://quotes.toscrape.com
https://httpstat.us/404
```

Expected: 2 successful scrapes, 1 broken link alert, 1 collection stats email.

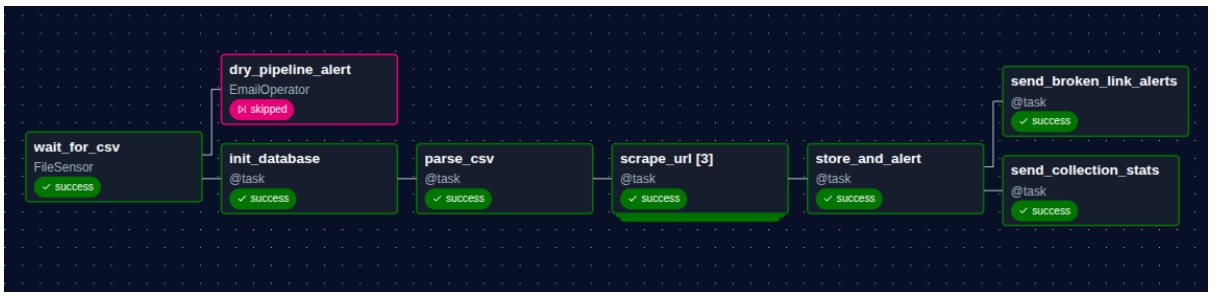


Figure 4: Test 1 — Successful DAG run. All tasks succeeded.

Result: All tasks completed successfully. Dynamic task mapping created 3 `scrape_url` instances (indices 0, 1, 2), all throttled through `scraping_pool`.

6.2 Test 2 — Broken Link Alert

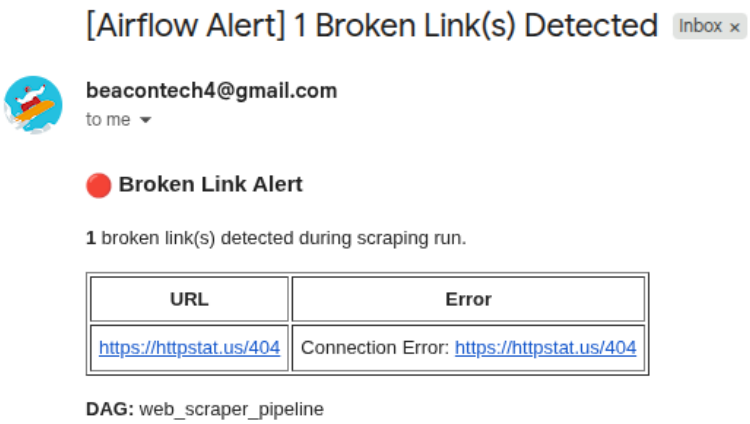


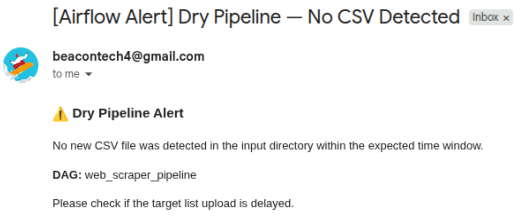
Figure 5: Test 2 — Broken link alert email received for `httpstat.us/404`

Result: Email correctly identified `httpstat.us/404` as a broken link and sent an HTML table with the URL and error description.

6.3 Test 3 — Dry Pipeline Alert

The `data/input/` directory was cleared of all CSV files before triggering the DAG. The FileSensor timed out after 2 minutes (configured `timeout=120`), marking itself as *Skipped*, which triggered `dry_pipeline_alert`.

(a) DAG run — sensor timed out



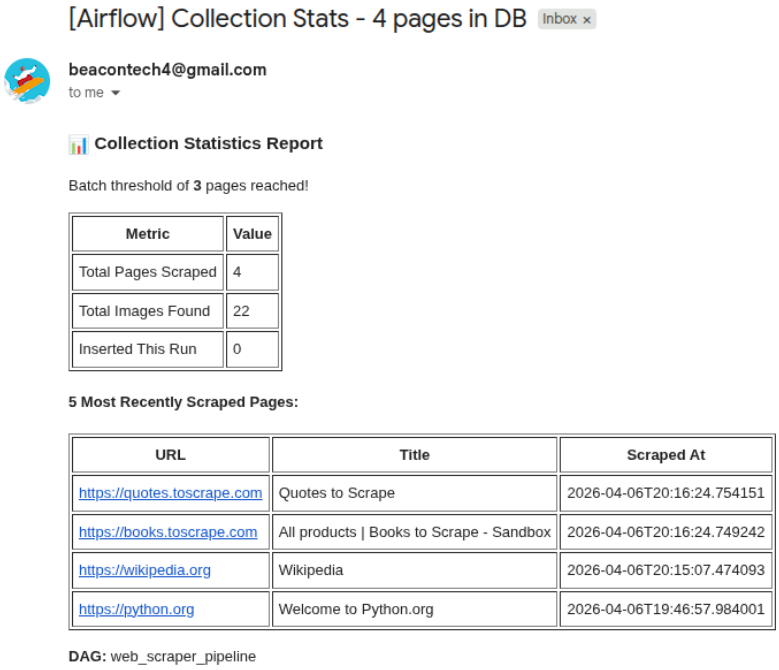
(b) Dry pipeline alert email

Figure 6: Test 3 — Dry Pipeline Alert

Table 3: Test 3 Task States

Task	State
wait_for_csv	Skipped (timeout)
dry_pipeline_alert	Success
init_database	Skipped
parse_csv	Skipped
scrape_url	Skipped
store_and_alert	Skipped
send_broken_link_alerts	Skipped
send_collection_stats	Skipped

6.4 Collection Statistics Email



[Airflow] Collection Stats - 4 pages in DB Inbox ×

beacontech4@gmail.com
to me

📊 **Collection Statistics Report**

Batch threshold of **3** pages reached!

Metric	Value
Total Pages Scraped	4
Total Images Found	22
Inserted This Run	0

5 Most Recently Scraped Pages:

URL	Title	Scraped At
https://quotes.toscrape.com	Quotes to Scrape	2026-04-06T20:16:24.754151
https://books.toscrape.com	All products | Books to Scrape - Sandbox	2026-04-06T20:16:24.749242
https://wikipedia.org	Wikipedia	2026-04-06T20:15:07.474093
https://python.org	Welcome to Python.org	2026-04-06T19:46:57.984001

DAG: web_scraper_pipeline

Figure 7: Collection statistics email showing 4 pages, 22 images in DB

7 Robustness & Error Handling

7.1 Retry Logic

All scraping tasks are configured with exponential backoff:

```
default_args = {
    "retries": 3,
    "retry_delay": timedelta(minutes=2),
    "retry_exponential_backoff": True,  # 2min -> 4min -> 8min
    "max_retry_delay": timedelta(minutes=10),
}
```

Listing 6: Retry configuration

7.2 Error Handling in Scraping

The `scrape_url` task handles four distinct failure modes:

Table 4: Error handling in scrape_url

Error Type	Handling	Alert
HTTP 404	Return `success=False`	Broken link email
Non-200 status	Return `success=False`	Broken link email
Timeout	Catch `requests.Timeout`	Broken link email
Connection error	Catch `ConnectionError`	Broken link email

7.3 Deduplication

SQLite's `INSERT OR IGNORE` with a `UNIQUE` constraint on the `url` column prevents duplicate entries across multiple pipeline runs.

8 Conclusion

This assignment successfully demonstrates a production-grade Airflow pipeline with all required components: file sensing, concurrent scraping with pool throttling, database persistence with deduplication, and a three-tier email alerting system.

Key technical challenges encountered and resolved:

- **Airflow 3.x API changes** — `one_skipped` trigger rule was removed; replaced with `all_skipped`
- **SmtpHook SSL issue** — Airflow 3.x `SmtpHook` had SSL configuration conflicts; resolved by using `smtplib` directly
- **Deprecated imports** — Migrated from `airflow.operators` to `airflow.providers.standard` namespace
- **Volume mounting** — Ensured `data/` and `scraped_data/` directories were mounted in `docker-compose.yaml`

The pipeline is fully containerised, self-contained within the repository, and reproducible via `docker compose up -d`.

A Repository Structure

```
assignment-6-muhammed786fiyas/
|-- dags/
|   `-- web_scraper_dag.py      # Main Airflow DAG
|-- data/input/
|   `-- .gitkeep                # Drop CSV files here
|-- scraped_data/
|   `-- .gitkeep                # SQLite DB stored here (gitignored)
|-- logs/                       # Auto-generated by Airflow
|-- config/                     # Auto-generated by Airflow
|-- plugins/
|   `-- .gitkeep
|-- docker-compose.yaml
|-- requirements.txt
|-- .env                        # Gitignored     contains SMTP_PASSWORD
|-- .gitignore
`-- README.md
```

B AI Disclosure

Tool Used

- **Claude** (Anthropic)

Nature of Prompts Provided

- Explanation of Apache Airflow concepts — FileSensor, Pools, Connections, CeleryExecutor, dynamic task mapping, and XCom.
- Debugging SMTP authentication and SSL errors — specifically the `SmtpHook.__init__()` unexpected keyword argument `files` error and the `SSLError: WRONG_VERSION_NUMBER` when using port 587.
- Conceptual explanation of the difference between Airflow Pools (concurrency limiters) and Connections (credential stores).
- Assistance generating the BeautifulSoup image URL extraction logic with handling for both relative and absolute URLs.
- Assistance structuring the `docker-compose.yaml` volume mounts for `data/` and `scraped_data/` directories.
- Requests to generate boilerplate code for SMTP email sending via `smtplib` with STARTTLS on port 587.

Sections Where AI Output Was Incorporated

- **Scraping logic** — BeautifulSoup image extraction with relative URL resolution was AI-assisted and manually verified against live websites.

- **docker-compose.yaml** — Initial volume mount additions for `data/` and `scraped_data/` were AI-assisted; SMTP environment variables were added and customised by the student.
- **Email sending code** — The `smtplib` STARTTLS pattern was AI-generated after the Airflow SmtpHook approach failed, and then replicated across all three alert tasks by the student.
- **DAG structure and task dependencies** (`>>` operators, Pool assignments, trigger rules) were **authored entirely by the student** as per the assignment's Code of Conduct.
- Assistance in structuring and improving the clarity of this written report.